



EJERCICIOS

Aplicar lo aprendido

6 ejercicios — del análisis al RTL simulable

Ejercicio 1 — Análisis — Blocking vs Non-blocking

ENUNCIADO

Predecir los valores de `a`, `b`, `c` tras el primer y segundo `posedge clk`, asumiendo `a=1`, `b=2`, `c=3` antes del primer flanco.

— Caso A (blocking) —

```
always_ff @(posedge clk) begin
  a = b; b = c; c = a;
end
```

— Caso B (non-blocking) —

```
always_ff @(posedge clk) begin
  a <= b; b <= c; c <= a;
end
```

¿Cuál implementa un rotador circular real?

DATOS

- Valores iniciales: `a=1`, `b=2`, `c=3`
- Clock: 50% duty, periodo 10 ns
- 2 flancos consecutivos
- Ambos `always_ff` (legal? — comentar)

ENTREGABLES

- Tabla con `a/b/c` en `t=0`, 10ns, 20ns para ambos casos
- ¿Cuál es el estilo correcto?
- ¿Cuál es el rotador circular?
- Bonus: simular ambos en iverilog y comparar

Ejercicio 2 — Registro 8-bit con Clock Enable

ENUNCIADO

Implementar en Verilog un registro de 8 bits con las siguientes características:

- Entrada de datos `d[7:0]` y salida `q[7:0]`
- Reset asíncrono activo-bajo (`rst_n`)
- Clock enable `ce` (activa-alta)
- Cuando `ce=0` el registro mantiene el valor anterior
- En reset, `q` se pone en `8'h00`

Acompañar con un testbench que aplique 10 vectores cubriendo: reset, escritura con `ce=1`, hold con `ce=0`, y cambio de valor mientras `ce=0` (no debe actualizarse).

DATOS

- Width = 8 bits
- `reset_n`: activo-bajo asíncrono
- `ce`: activo-alto sync
- Clock: 100 MHz (periodo 10 ns)
- Patrón: `reg_ce` parametrizable

ENTREGABLES

- `reg_ce.v` — módulo sintetizable
- `tb_reg_ce.v` — testbench con `$display`
- `run.sh` — script iverilog + vvp
- Reporte: PASS/FAIL en todos los vectores

Ejercicio 3 — Contador BCD 0-9 con Enable

ENUNCIADO

Diseñar un contador decimal (BCD, 0-9) que:

- Incrementa en cada flanco de clock cuando `en=1`
- Cuenta de 0 a 9 y vuelve a 0 (rollover)
- Emite una salida `pulse tc` (terminal count) que vale 1 durante UN ciclo cuando el contador pasa de 9 a 0
- Reset síncrono activo-alto pone el contador en 0

El testbench debe verificar el rollover y el comportamiento del pulso `tc` comparando contra un golden model en el propio testbench.

DATOS

- Width = 4 bits (suficiente para 0-9)
- Reset síncrono activo-alto
- Frecuencia: 100 MHz
- `tc` debe ser 1 ciclo, NO 2

ENTREGABLES

- `bcd_counter.v`
- `tb_bcd_counter.v` con auto-check
- Verificar que tras 100 ciclos con `en=1`, el contador valió exactamente 10 veces el ciclo `0→9→0→...`
- Generar VCD para inspección en GTKWave

💡 *`tc` se asigna como salida combinacional: `assign tc = (cnt == 4'd9) && en`. NO la registres, sino se desfasa 1 ciclo.*

Ejercicio 4 — FSM Detector de Secuencia "101"

ENUNCIADO

Implementar una FSM Moore que detecte la secuencia binaria "1 0 1" en una entrada serial x . Cuando se detecta el patrón:

- La salida y se pone en 1 durante UN ciclo de clock
- El detector se solapa: el último "1" de una detección puede ser el primer "1" de la siguiente

Ejemplo: $x = 1\ 1\ 0\ 1\ 0\ 1 \rightarrow y = 0\ 0\ 0\ 1\ 0\ 1$ (dos detecciones, ciclos 4 y 6)

Mínimo: usar 4 estados ($S0$, $S1$, $S10$, $S101$). Diagrama de transiciones obligatorio en el README.

DATOS

- 1 bit de entrada, 1 bit de salida
- Reset asíncrono activo-bajo
- Codificación: a elección (binary, one-hot)
- Patrón Moore: $y = f(\text{estado})$, no $f(x)$

ENTREGABLES

- detector_101.v (3 always: registro + próximo-estado + salida)
- tb_detector_101.v con secuencia "11010110101"
- README.md con el diagrama de estados (ASCII art o markdown)
- Verificación: contar detecciones esperadas vs obtenidas

Ejercicio 5 — Pipeline 3-etapas con Valid/Ready

ENUNCIADO

Implementar un pipeline de 3 etapas que calcula:

$$y = ((x + A) * B) \gg 4$$

Etapla 1: suma $x + A$

Etapla 2: multiplica el resultado por B

Etapla 3: shift right aritmético 4 posiciones

La interfaz usa handshake AXI-Stream:

- Entrada: x_in , $valid_in$, $ready_out$
- Salida: y_out , $valid_out$, $ready_in$
- Cuando $ready_in=0$, el pipeline DEBE bloquear (stall). Cuando hay stall y llega $valid_in=1$, $ready_out=0$.

DATOS

- $A=8'sd5$, $B=8'sd3$ (constantes hardcoded)
- x_in : 8 bits signed
- y_out : 16 bits signed
- Latencia: 3 ciclos
- Throughput: 1 muestra/ciclo en régimen sin stalls

ENTREGABLES

- `pipeline_3stage.v`
- `tb_pipeline.v` con golden model en software
- Probar con stall ($ready_in$ alternando 1/0)
- Reportar throughput observado vs teórico

💡 Cada etapa tiene su propio valid registrado. $ready_out = ready_in$ (la lógica de back-pressure se propaga). Cuando $ready_in=0$, congelar todos los FFs con un CE común.

Ejercicio 6 — Debugging — Encontrar el latch oculto

ENUNCIADO

El siguiente código tiene un BUG sutil que la síntesis convierte en latch:

```
always_comb begin
  case (op)
    2'b00: y = a + b;
    2'b01: y = a - b;
    2'b10: y = a & b;
  endcase
end
```

Tareas:

- Identificar el bug y por qué se infiere latch
- Proponer DOS fixes (default vs pre-asignación)
- Implementar ambas versiones y verificar equivalencia
- Inspeccionar con yosys que no quedan latches

DATOS

- y, a, b: signed [7:0]
- op: 2 bits — solo 3 valores definidos
- Con op=2'b11 sin asignar → la herramienta infiere latch para mantener valor previo

ENTREGABLES

- alu_bad.v alu_fix1.v alu_fix2.v
- tb_alu.v compara las 3 versiones con 256 vectores
- report.md con análisis del bug y comparativa
- yosys script para verificar (opcional)